Python Engineering - EX23

WEEK 04 | CLASS 15 - Exceptions & Debugging | Safe File Loading

Objetivo: aprender a detectar, interpretar y gestionar errores previsibles al leer datos de ingenieria. Trabajaremos con un CSV correcto, uno inexistente y uno corrupto.

> La meta no es conseguir que 'nunca haya errores'. La meta es que, cuando algo falla, sepamos **que fallo, donde fallo, por que fallo y que debe hacer el programa**.

Ruta exacta

```
01_python_bootcamp/
`-- week_04_oop_files_testing/
    `-- class_15_exceptions_debugging/
        |-- notes/
        `-- exercises/
            `-- ex23_safe_file_loading/
                |-- file_io.py
                |-- main.py
                `-- data/
                    |-- valid_trajectory.csv
                    `-- corrupted_trajectory.csv
```

Los tres escenarios

Caso	Que ocurre	Excepcion esperada
Valid file	Archivo correcto y columnas numericas	Ninguna
Missing file	La ruta apunta a un archivo inexistente	FileNotFoundError
Corrupted file	Una fila no cumple el formato esperado	ValueError

Archivos de referencia

`valid_trajectory.csv`:

```
h_m,v_m_s,t_s
100.000000,20.000000,0.000000
102.000000,19.019000,0.100000
103.901900,18.038000,0.200000
```

`corrupted_trajectory.csv`:

```
h_m,v_m_s,t_s
100.000000,20.000000,0.000000
102.000000,NOT_A_NUMBER,0.100000
103.901900,18.038000,0.200000
```

BLOCK A - Cargar el CSV sin ocultar errores

En `file_io.py` empezamos con una funcion parecida a EX22, pero ahora validamos explicitamente el contrato del archivo.

```
import numpy as np

def load_trajectory_csv(file_path):

    rows = []

    with open(file_path, 'r', encoding='utf-8') as file:
        header = file.readline().strip()

        if header != 'h_m,v_m_s,t_s':
            raise ValueError(
                'Invalid CSV header'
            )

        for line_number, line in enumerate(file, start=2):
```

```
            clean_line = line.strip()

            if clean_line == '':
                continue

            values = clean_line.split(',')

            if len(values) != 3:
                raise ValueError(
                    f'Line {line_number}: expected 3 columns'
                )

            h = float(values[0])
            v = float(values[1])
            t = float(values[2])

            rows.append([h, v, t])

    return np.array(rows)
```

Observa algo importante: esta funcion **no intenta arreglar silenciosamente** un archivo incorrecto. Si el contrato no se cumple, genera una excepcion clara.

BLOCK B - raise ValueError(...)

`raise` significa: 'esta situacion no es valida; detiene este camino normal y crea una excepcion'.

```
if len(values) != 3:
    raise ValueError(
        f'Line {line_number}: expected 3 columns'
    )
```

No usamos `raise` porque Python haya fallado solo. Lo usamos nosotros para hacer cumplir una regla del formato.

BLOCK C - enumerate(..., start=2)

```
for line_number, line in enumerate(file, start=2):
    ...
```

La primera linea del archivo es la cabecera. Por eso la primera fila de datos es fisicamente la linea 2. Guardar ese numero en el error hace el debugging mucho mas util.

BLOCK D - Gestionar excepciones desde main.py

La funcion de I/O detecta problemas. `main.py` decide como responder.

```
from pathlib import Path
from file_io import load_trajectory_csv

def test_file(file_path):

    print(f'\nTesting: {file_path.name}')

    try:
        trajectory = load_trajectory_csv(file_path)

    except FileNotFoundError as error:
        print('FILE ERROR')
        print(type(error).__name__)
        print(error)

    except ValueError as error:
        print('DATA ERROR')
        print(type(error).__name__)
        print(error)

    else:
        print('LOAD OK')
        print(f'Shape: {trajectory.shape}')
        print(trajectory)

    finally:
        print('Test finished')
```

BLOCK E - Entender el flujo try / except / else / finally

```
try
 |
 +-- success --------------------> else
 |
 +-- FileNotFoundError ----------> matching except
 |
 +-- ValueError -----------------> matching except
 |
 v
finally runs in every case
```

try: codigo que puede fallar. **except:** respuesta a una excepcion concreta. **else:** solo si try termina correctamente. **finally:** se ejecuta tanto con exito como con error.

BLOCK F - Crear las tres rutas

```
base_dir = Path(__file__).resolve().parent
data_dir = base_dir / 'data'

valid_file = data_dir / 'valid_trajectory.csv'
missing_file = data_dir / 'missing_trajectory.csv'
corrupted_file = data_dir / 'corrupted_trajectory.csv'
```

No crees `missing_trajectory.csv`. Queremos precisamente que no exista.

BLOCK G - Ejecutar los tres casos

```
def main():

    base_dir = Path(__file__).resolve().parent
    data_dir = base_dir / 'data'

    files_to_test = [
        data_dir / 'valid_trajectory.csv',
        data_dir / 'missing_trajectory.csv',
        data_dir / 'corrupted_trajectory.csv'
    ]

    for file_path in files_to_test:
        test_file(file_path)

if __name__ == '__main__':
    main()
```

Resultados esperados

```
Testing: valid_trajectory.csv
LOAD OK
Shape: (3, 3)
...
Test finished

Testing: missing_trajectory.csv
FILE ERROR
FileNotFoundError
...
Test finished

Testing: corrupted_trajectory.csv
DATA ERROR
ValueError
could not convert string to float: 'NOT_A_NUMBER'
Test finished
```

BLOCK H - Debugging deliberado con VS Code

Haz una segunda ejecucion del archivo corrupto usando el debugger.

```
Suggested breakpoints:

1. values = clean_line.split(',')
2. v = float(values[1])
```

```
3. except ValueError as error:
```

En el panel de variables observa:

```
line_number
clean_line
values
values[1]
type(values[1])
```

Antes de ejecutar `float(values[1])`, intenta predecir que ocurrira. El debugger no es solo para mirar despues del error; tambien sirve para seguir el flujo antes de que ocurra.

BLOCK I - Leer un traceback

Comenta temporalmente el `except ValueError` y ejecuta el archivo corrupto para dejar que Python muestre el traceback completo. Luego vuelve a activar el except.

Busca tres cosas:

```
1. Exception type
2. Exception message
3. Last relevant line of your own code
```

Normalmente empezaremos leyendo un traceback **desde abajo hacia arriba**: abajo esta el tipo/mensaje y justo encima suele aparecer la linea donde se produjo el fallo.

FINAL VALIDATION

```
Explain in comments:
- What is an exception?
- How is an exception different from a syntax error?
- What code should go inside try?
- Why should we catch FileNotFoundError and ValueError separately?
- What does 'as error' give us?
- When does else execute?
- When does finally execute?
- What does raise do?
- Why do we raise ValueError for an invalid header/column count?
- Why should we avoid except Exception: in this exercise?
- What does enumerate(file, start=2) give us?
- Why is a line number useful in an error message?
- What is a traceback?
- Which part of a traceback would you inspect first?
- Why is silently returning an empty array after corrupted data dangerous?
- What is the difference between handling an error and hiding an error?
```

Criterio de cierre

EX23 estara completado cuando puedas provocar voluntariamente los tres casos, predecir que bloque se ejecutara, explicar por que aparece cada excepcion y localizar el dato problematico con el debugger o el traceback.